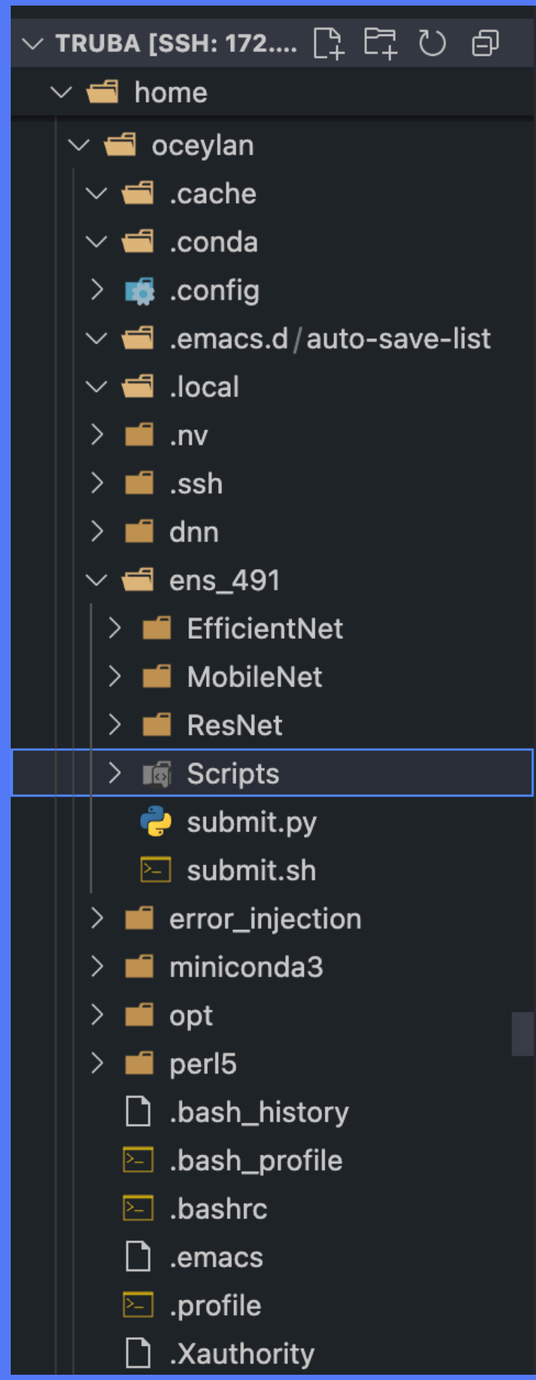
TensorRT

/truba Dosya Sistemi Erişimi

- ARF ACC ortamında, `/truba` dosya sistemine erişim **sadece cuda-ui kullanıcı arayüzü** üzerinden mümkündür.
- Kolyoz ve palamut hesaplama sunucularında doğrudan `/truba` dosya sistemine erişim **bulunmamaktadır**.
- Hesaplama işlerinizi ve veri transferlerinizi planlarken bu erişim kısıtını göz önünde bulundurunuz.

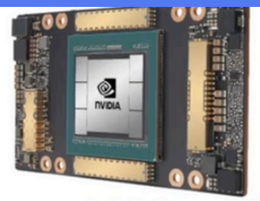
/arf Dosya Sistemi (WEKA Yüksek Performanslı Dosya Sistemi)

- `/arf` dosya sistemi, WEKA tabanlı yüksek performanslı paralel dosya sistemi olarak yapılandırılmıştır.
- Tüm hesaplama sunucuları (kolyoz ve palamut dahil) ve kullanıcı arayüzleri tarafından erişilebilir durumdadır.
- Hesaplama sırasında yüksek I/O performansı ve veri güvenliği için tüm işlerinizde bu dosya sistemini kullanmanız önerilir.



PyTorch

Torch-TensorRT



A100

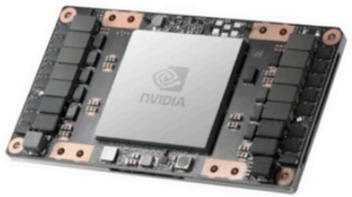
JETSON AGX XAVIER

A30

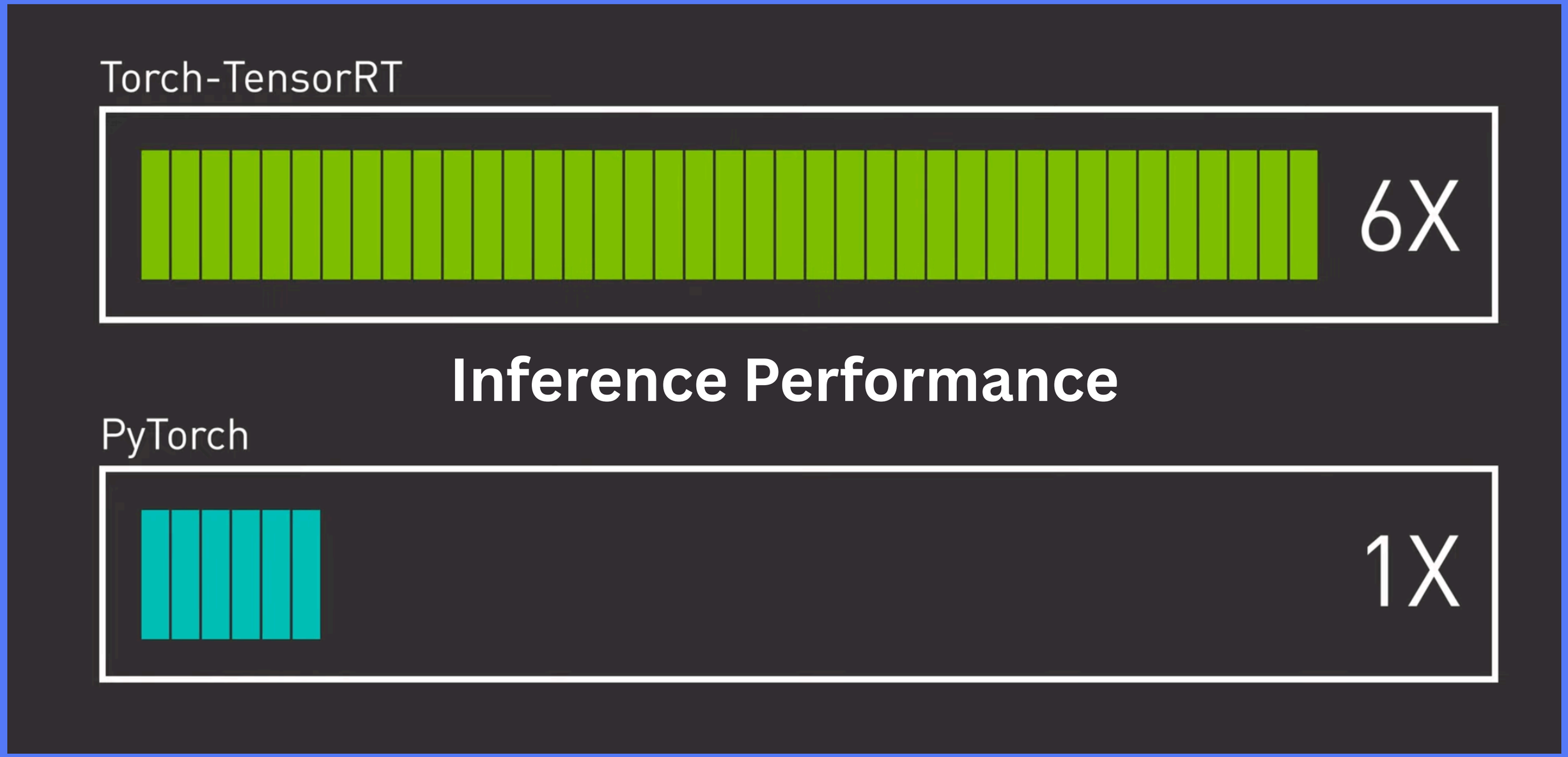
JETSON XAVIER

A40

NVIDIA DLA

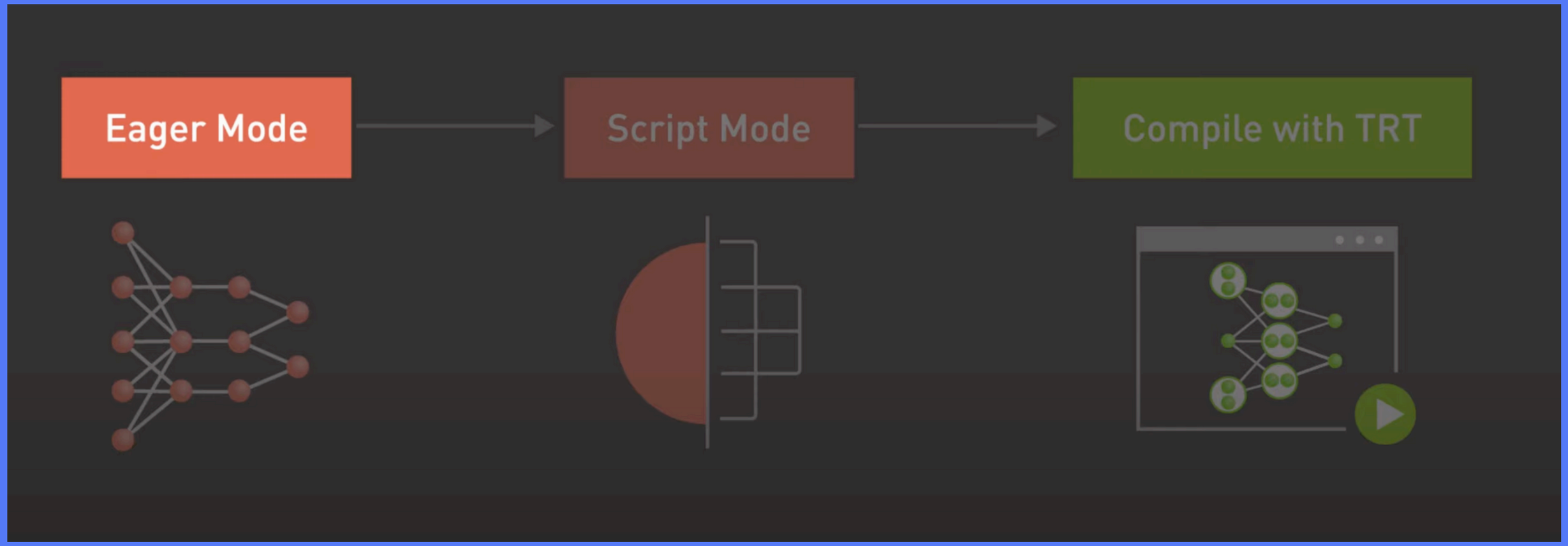


DRIVE



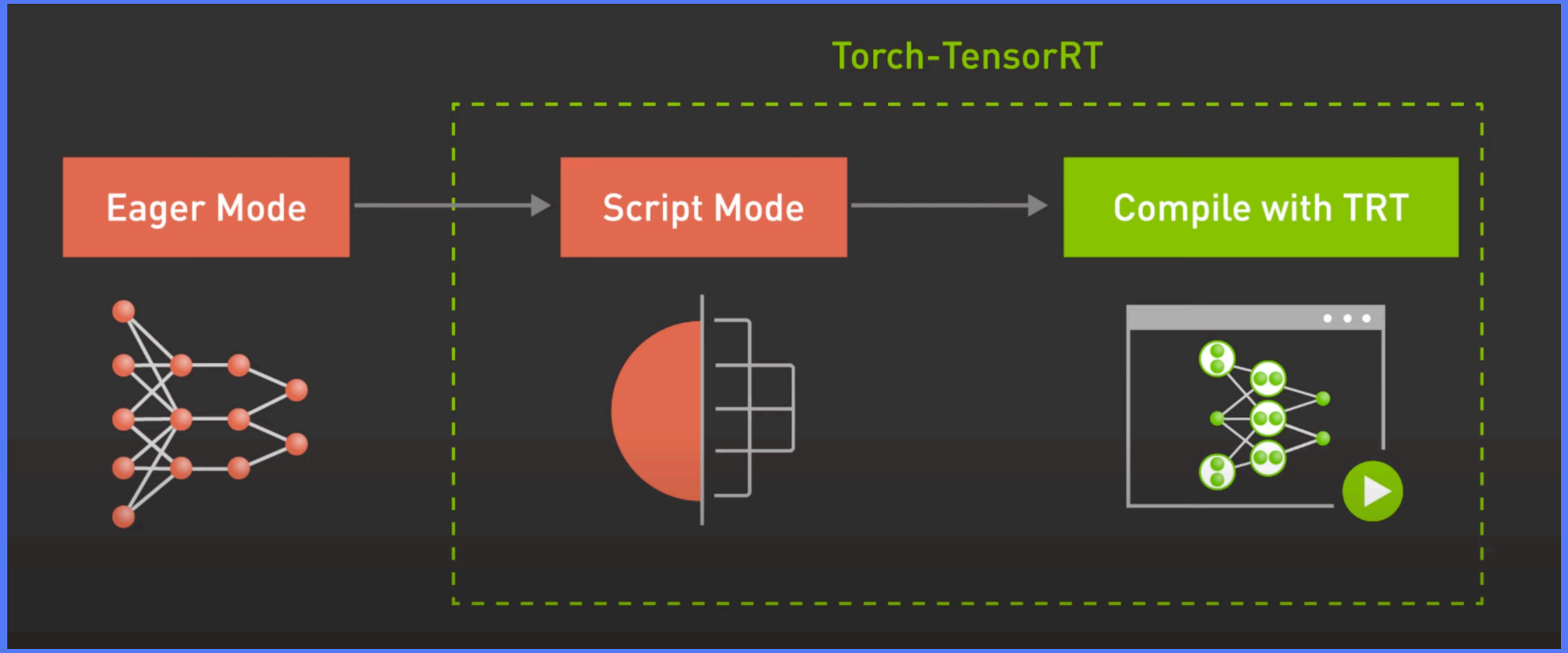
Modelin yeni veriler üzerinde ne kadar hızlı ve doğru tahmin yaptığıdır.

PyTorch has 2 modes



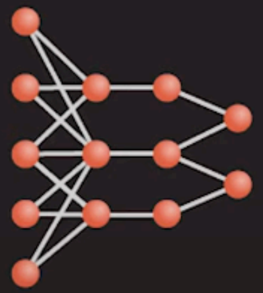
Eager is for training

Script mode - also known as graph mode



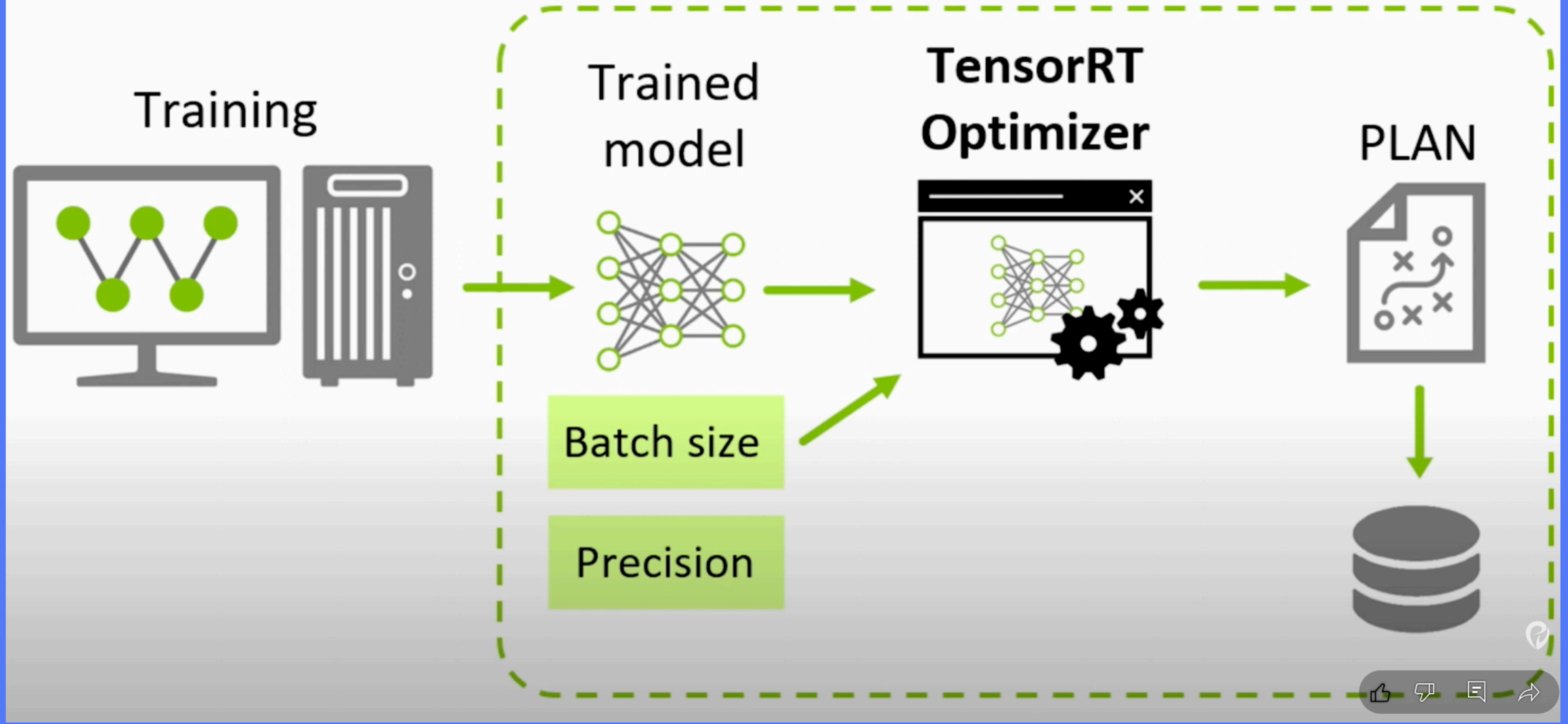
for inference of models

Eager Mode



```
trt_module = torch_tensorrt.compile(model, inputs=[example_input])
```

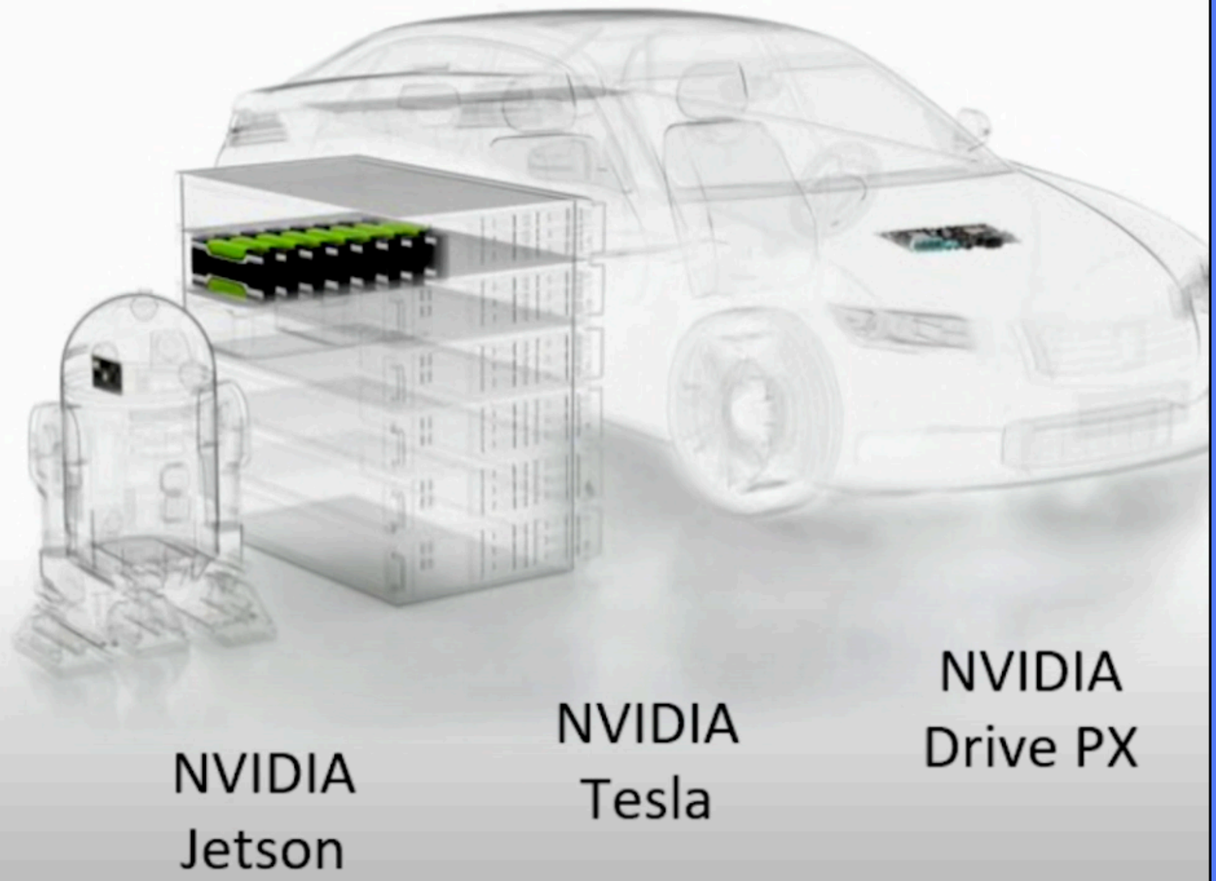
Step 1: Optimization



Step 2: Inference

PLAN

TensorRT Runtime

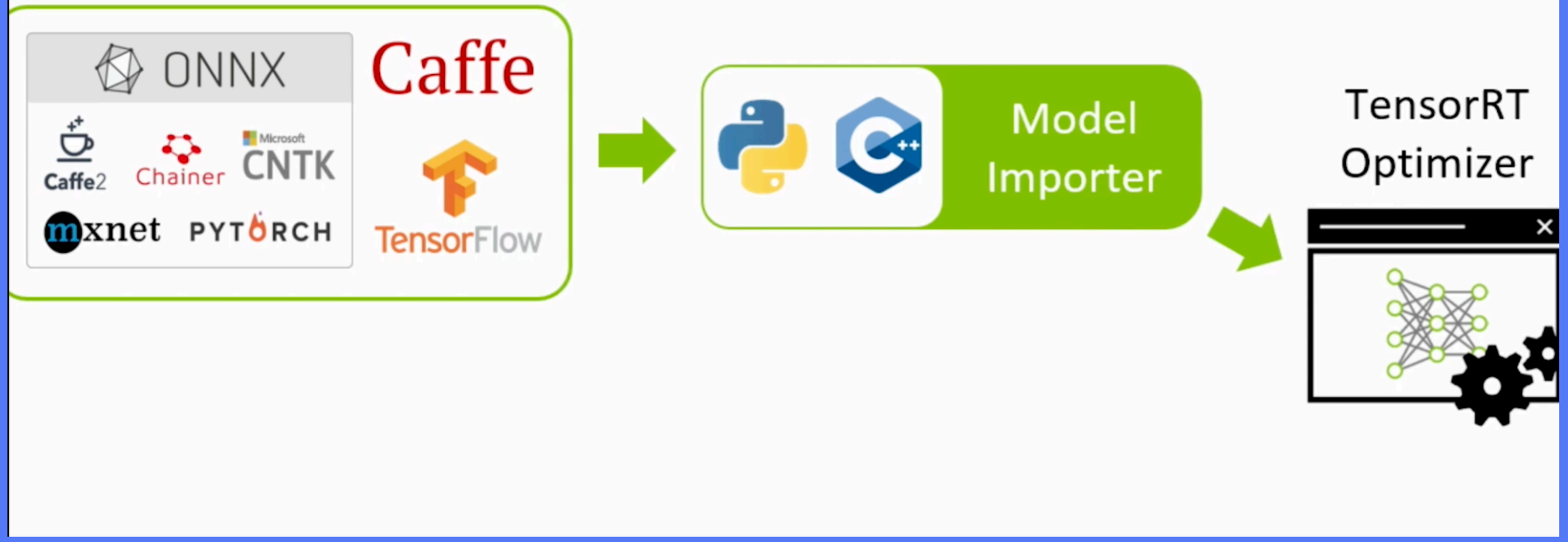


NVIDIA Jetson

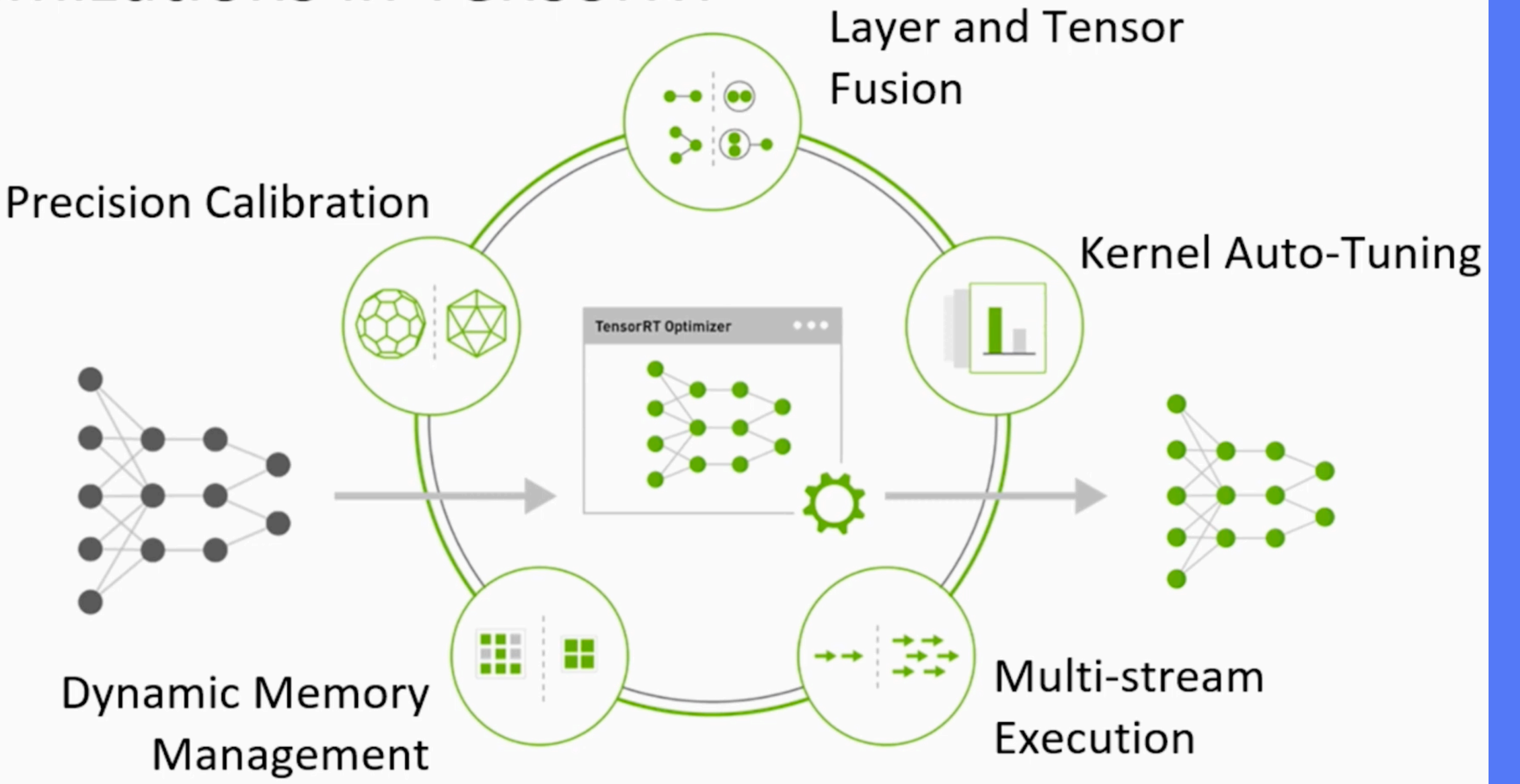
NVIDIA Tesla

NVIDIA Drive PX

Model import

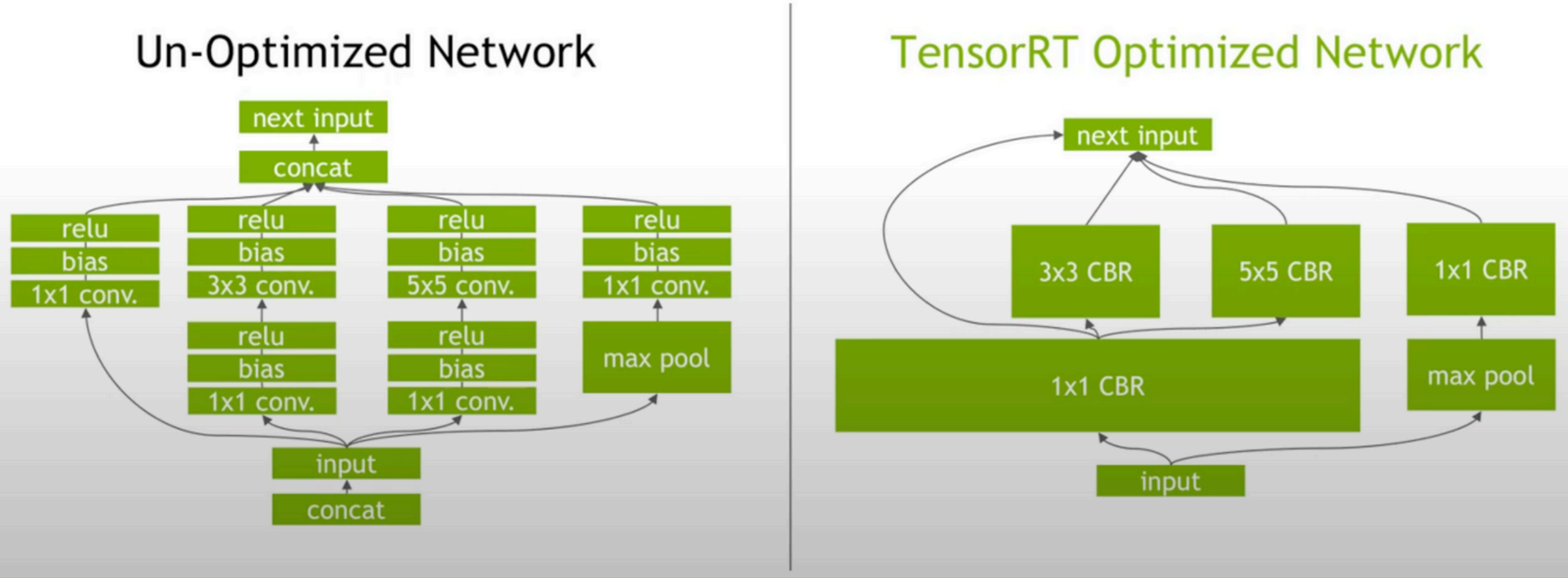


Optimizations in TensorRT

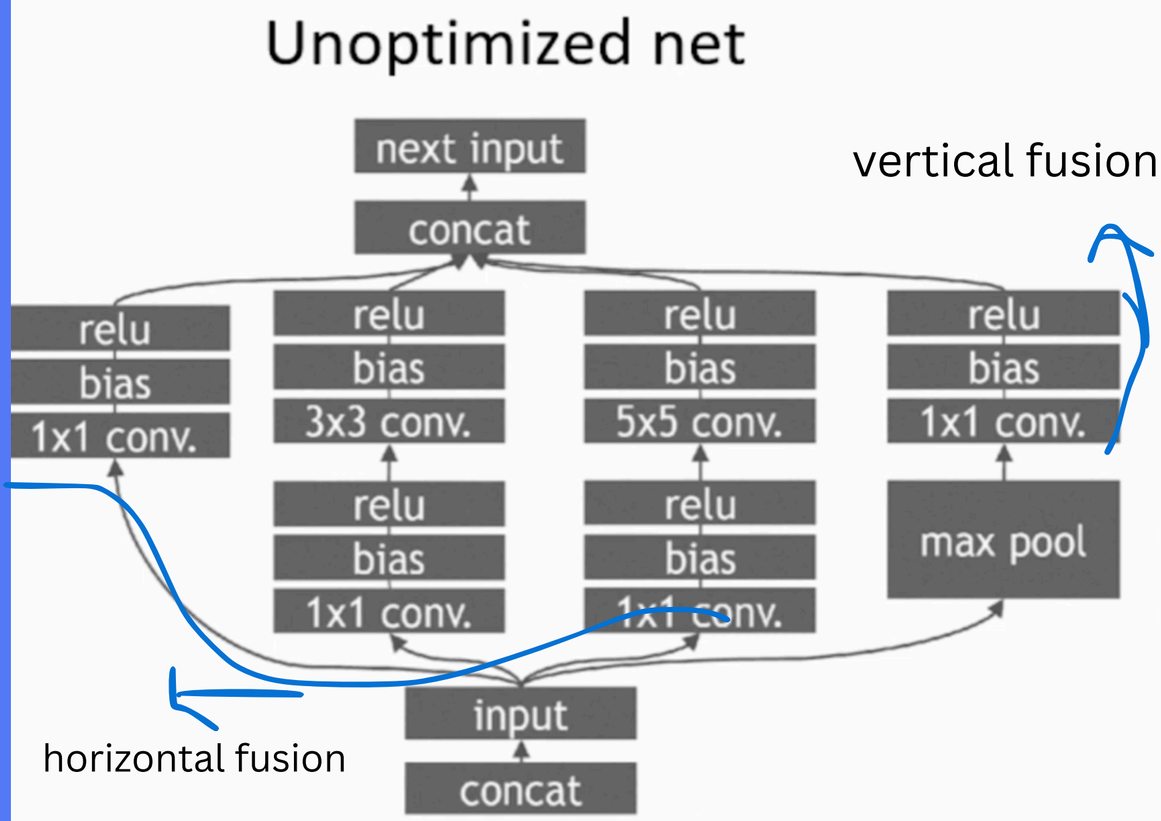


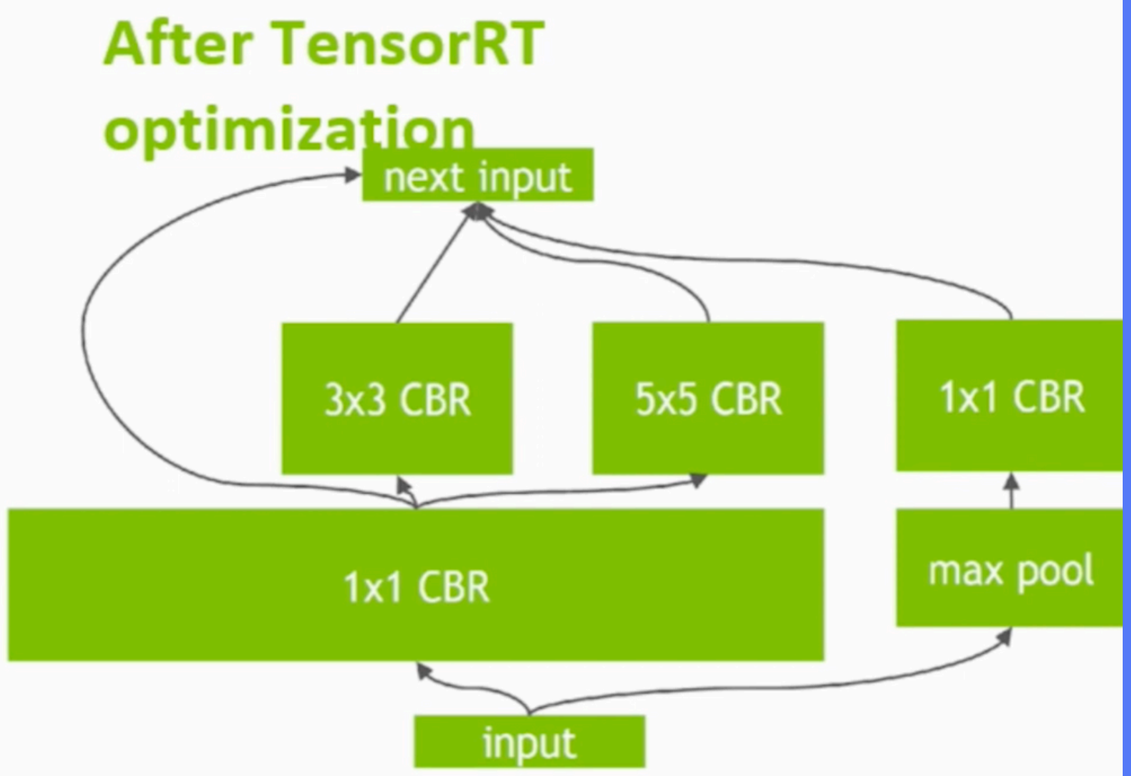
1. KERNEL FUSION

- Improve GPU utilization - less kernel launch overhead, better memory usage and bandwidth
- Vertical fusion = Combine sequential kernel calls
- Horizontal fusion = Combine same kernels that have common input but different weights



Layer and tensor fusion





- Vertical + horizontal fusion
- Elimination of concat layers

concat layers only make memory manipulation they do not make any calculation

2. PRECISION CALIBRATION

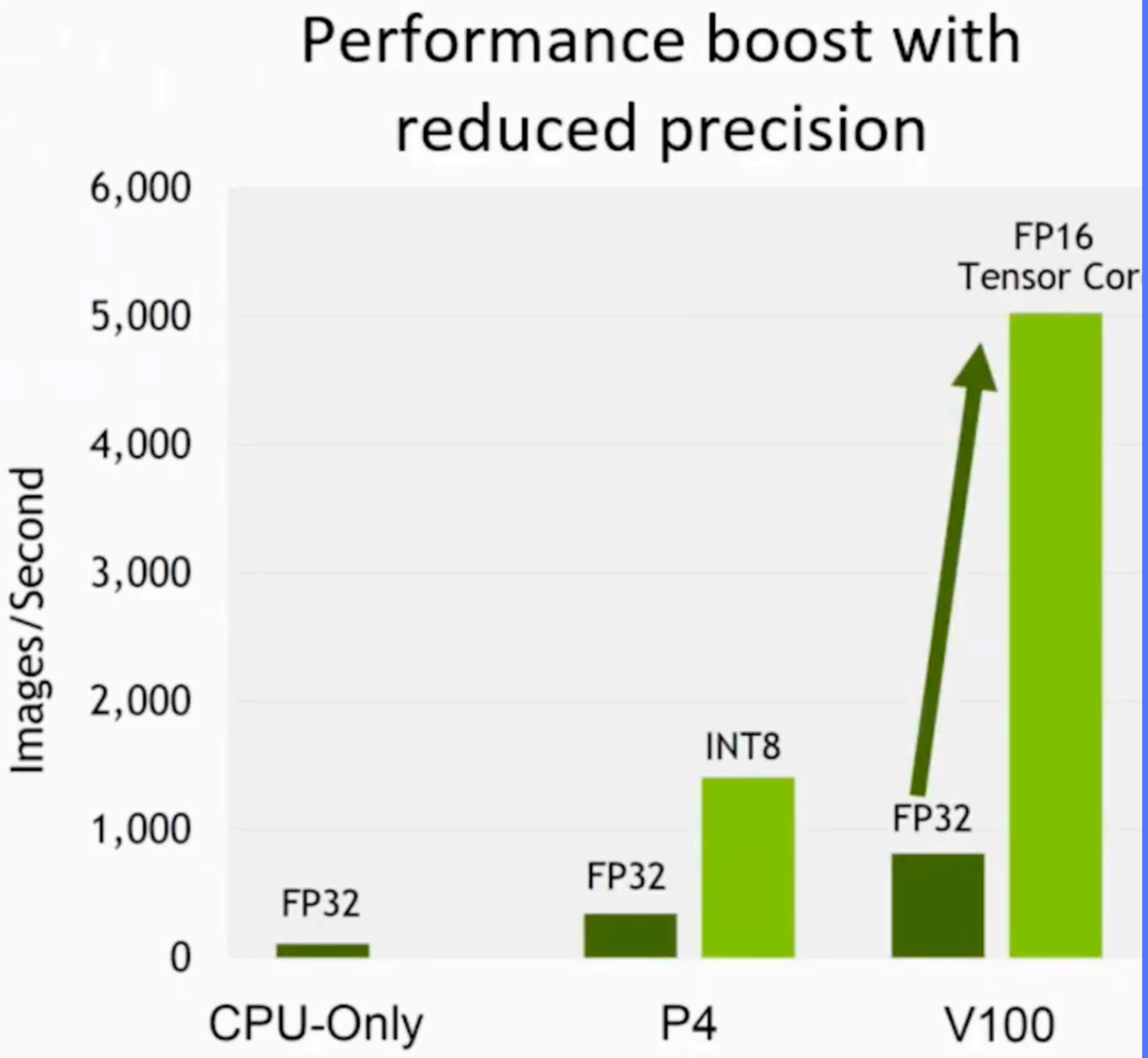
- DNNs often trained with full precision (FP32)
- Inference does not require backpropagation – allows using lower precisions via a calibration step
- Automated parameter-free calibration step using a representative sample
- Smaller model size, lower memory utilization and latency, higher throughput

Precision	Dynamic Range
FP32	-3.4×10^{38} ~ $+3.4\times10^{38}$
FP16	-65504 ~ +65504
INT8	-128 ~ +127

Precision calibration

Precision	Range
FP32	$-3.4 \times 10^{38} \sim +3.4 \times 10^{38}$
FP16	-65504 ~ +65504
INT8	-128 ~ +127

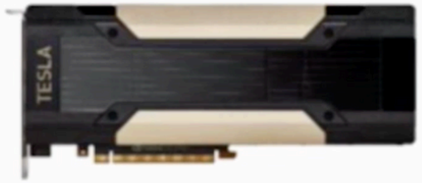
Automatic weights calibration is performed, if **INT8** is used.



3. KERNEL AUTO-TUNING

- There are multiple low-level algorithms/implementations for common operations
- TensorRT selects the optimal kernels based on your parameters e.g. batch size, filter-size, input data size
- TensorRT selects the optimal kernel based on your target platform

Automatic selection of best matching kernels



Tesla V100

Jetson TX2

Drive PX2

- Batch size
- Input size
- Size of all buffers

Hundreds of specialize kernels for various GPUs and hyperparameters

4. DYNAMIC TENSOR MEMORY

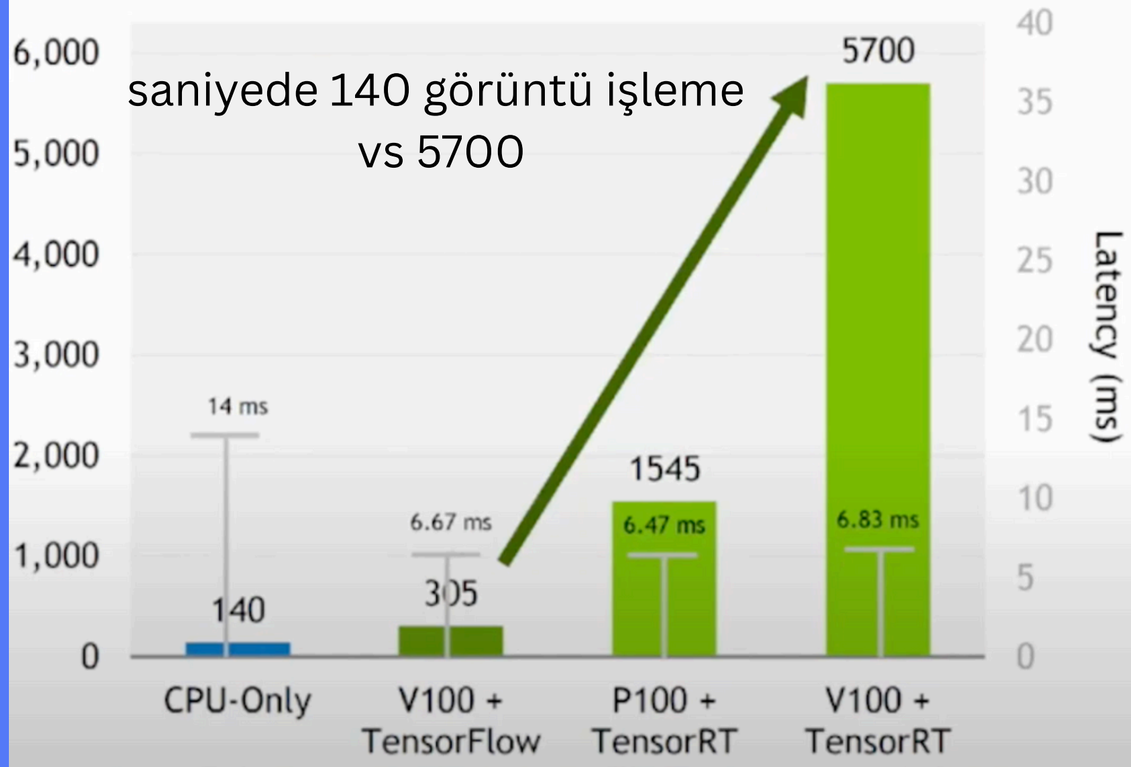
- Allocates just the memory required for each tensor and only for the duration of its usage
- Reduces memory footprint and improves memory re-use

5. MULTI-STREAM EXECUTION

- Allows processing multiple input streams in parallel

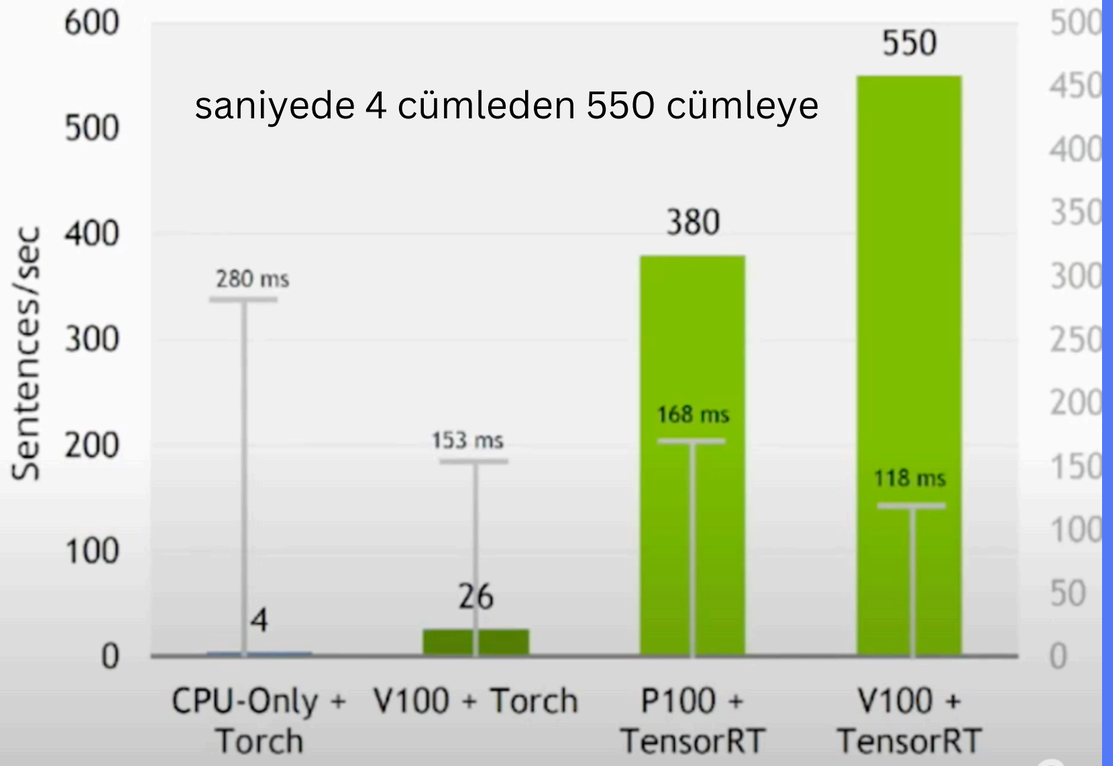
TensorRT Performance

40x Faster CNNs on V100 vs. CPU-Only Under 7ms Latency (ResNet50)



Inference throughput (images/sec) on ResNet50. **V100 + TensorRT**: NVIDIA TensorRT (FP16), batch size 39, Tesla V100-SXM2-16GB, E5-2690 v4@2.60GHz 3.5GHz Turbo (Broadwell) HT On. **P100 + TensorRT**: NVIDIA TensorRT (FP16), batch size 10, Tesla P100-PCIE-16GB, E5-2690 v4@2.60GHz 3.5GHz Turbo (Broadwell) HT On **V100 + TensorFlow**: Preview of volta optimized TensorFlow (FP16), batch size 2, Tesla V100-PCIE-16GB, E5-2690 v4@2.60GHz 3.5GHz Turbo (Broadwell) HT On. **CPU-Only**: Intel Xeon-D 1567 Broadwell-E CPU and Intel DL SDK. Score doubled to comprehend Intel's stated claim of 2x performance improvement on Skylake with AVX512.

140x Faster Language Translation RNNs on V100 vs. CPU-Only Inference (OpenNMT)



Inference throughput (sentences/sec) on OpenNMT 692M. **V100 + TensorRT**: NVIDIA TensorRT (FP32), batch size 64, Tesla V100-PCIE-16GB, E5-2690 v4@2.60GHz 3.5GHz Turbo (Broadwell) HT On. **P100 + TensorRT**: NVIDIA TensorRT (FP32), batch size 64, Tesla P100-PCIE-16GB, E5-2690 v4@2.60GHz 3.5GHz Turbo (Broadwell) HT On **V100 + Torch**: Torch (FP32), batch size 4, Tesla V100-PCIE-16GB, E5-2690 v4@2.60GHz 3.5GHz Turbo (Broadwell) HT On. **CPU-Only**: (orch (FP32), batch size 1, Intel E5-2690 v4@2.60GHz 3.5GHz Turbo (Broadwell) HT On

QUANTIZED NETWORK ACCURACY

Classification

Model	FP32	FP16	INT8
MobileNet v1	71.01	70.96	69.43
MobileNet v2	74.08	74.08	73.85
NASNet (large)	82.72	82.71	82.66
NASNet (mobile)	73.97	73.7	73.4
RN50 (v1.5)	76.56	76.57	76.47
RN152 (v2)	78.45	78.45	77.95
VGG-16	70.89	70.91	70.82
VGG-19	71.01	70.92	70.85
Inception v3	77.99	77.98	77.85

All results percentage accuracy on Imagenet validation set

Recurrent (translation)

Model	FP32	INT8
GNMT (8 Layer)	29.89	29.97

BLEU Score for DE -> EN translation on WMT newstest-2015

Detection (COCO Train 2017)

Model	Backbone	FP32	FP16	INT8
SSD-300	MobileNet v1	26	26	25.8
SSD-300	MobileNet v2	27.4	27.4	26.8
Faster RCNN	RN-101	33.7		33.4

All results COCO mAP on COCO 2017 validation

Detection (VOC Train+Val 07+12)

Model	Backbone	FP32	FP16	INT8
SSD-300	VGG-16	77.7	77.7	77.6
SSD-512	VGG-16	79.9	79.9	79.9

All results VOC mAP on VOC 07 test

📊 === Batch Size GPU Benchmark Results ===

Batch Size	Total Images	Total Time (s)	Avg Batch Time (s)	Avg Image Time (ms)	Throughput (img/sec)	Top-1 Acc (%)	Top-5 Acc (%)
16	50000	224.59	0.0719	4.49	222.6	69.06	88.52
32	50000	137.11	0.0877	2.74	364.7	69.06	88.52
64	50000	85.74	0.1096	1.71	583.2	69.06	88.52
128	50000	52.57	0.1345	1.05	951.1	69.06	88.52
256	50000	36.57	0.1866	0.73	1367.4	69.06	88.52
512	50000	30.04	0.3065	0.60	1664.5	69.06	88.52
1024	50000	24.67	0.5035	0.49	2026.6	69.06	88.52

```
=== Starting TensorRT Benchmark ===

🚀 Running TensorRT (FP16) with batch size = 16...
Found 50000 files belonging to 1000 classes.
TensorRT batch 16: 100%|| 3125/3125 [13:21<00:00,  3.90batch/s]

🚀 Running TensorRT (FP16) with batch size = 32...
Found 50000 files belonging to 1000 classes.
TensorRT batch 32: 100%|| 1563/1563 [09:54<00:00,  2.63batch/s]

🚀 Running TensorRT (FP16) with batch size = 64...
Found 50000 files belonging to 1000 classes.
TensorRT batch 64: 100%|| 782/782 [07:21<00:00,  1.77batch/s]

🚀 Running TensorRT (FP16) with batch size = 128...
Found 50000 files belonging to 1000 classes.
TensorRT batch 128: 100%|| 391/391 [05:21<00:00,  1.21batch/s]

🚀 Running TensorRT (FP16) with batch size = 256...
Found 50000 files belonging to 1000 classes.
TensorRT batch 256: 100%|| 196/196 [04:21<00:00,  1.34s/batch]

🚀 Running TensorRT (FP16) with batch size = 512...
Found 50000 files belonging to 1000 classes.
TensorRT batch 512: 100%|| 98/98 [03:21<00:00,  2.06s/batch]

🚀 Running TensorRT (FP16) with batch size = 1024...
Found 50000 files belonging to 1000 classes.
TensorRT batch 1024:   4%|                                                | 2/49 [00:17<07:35,  9.68s/batch]
Killed
```